Day 77 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 12, Day 1 reference

Watch: Positional embeddings in transformers EXPLAINED (AI Coffee Break with Letitia) | Positional Encoding in Transformer Neural Networks Explained (CodeEmporium)

`day77_positional_encoding.py`, segment by segment, in the order `main()` uses things. Day 76's attention functions and Day 62's data code are copied verbatim and explained there. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: p = position · i = pair index (columns 2i and 2i+1) · d = vector size · PE = the encoding table · R_k = the rotation for a shift of k · E[w] = word w's vector

Segment 0 — Setup

Settings

CODE

```
SEEDS = [0, 1, 2]
EPOCHS = 25
LR = 2.0
HEADS = 4
SCALES = (0.3, 1.0, 3.0, 10.0)
TOY_V = 10
TOY_LEN = (8, 16)
TOY_STEPS = 3000
MAX_LEN = 128
```

- `SCALES = (0.3, 1.0, 3.0, 10.0)` — the strengths of the encoding tried on validation.
- `TOY_LEN = (8, 16)` — a tuple read as (shortest, longest) toy training length.
- `MAX_LEN = 128` — the table is built once for 128 positions; every sentence uses its first T rows.

Segment 1 — The encoding

sinusoidal_encoding

MATH

$$angle[p, i] = p / 10000^{2i/d};\quad PE[p, 2i] = sin(angle),\quad PE[p, 2i+1] = cos(angle)$$

CODE

```
def sinusoidal_encoding(n, d):
    p = np.arange(n)[:, None]
    i = np.arange(d // 2)[None, :]
    angle = p / 10000 ** (2 * i / d)
    PE = np.zeros((n, d))
    PE[:, 0::2] = np.sin(angle)
    PE[:, 1::2] = np.cos(angle)
    return PE
```

- `np.arange(n)[:, None]` — a column of positions; np.arange(d // 2)[None, :] a row of pair indices.
- `p / 10000 ** (2 * i / d)` — broadcasting a column against a row gives an (n, d/2) table of angles.
- `PE[:, 0::2]` — a step slice: every second column from 0 (0, 2, 4, ...); 1::2 starts from 1.

shift_rotation

MATH

$$[\sin x, \cos x]\ [[\cos a, -\sin a], [\sin a, \cos a]] = [\sin(x + a), \cos(x + a)]$$

CODE

```
def shift_rotation(k, d):
    R = np.zeros((d, d))
    for i in range(d // 2):
        a = k / 10000 ** (2 * i / d)
        R[2 * i:2 * i + 2, 2 * i:2 * i + 2] = [[np.cos(a), -np.sin(a)], [np.sin(a), np.cos(a)]]
    return R
```

- `R[2 * i:2 * i + 2, 2 * i:2 * i + 2] = [[...], [...]]` — assigns a 2 x 2 block on the diagonal from a nested list.
- `np.cos(a), -np.sin(a)` — the block turns the row vector (sin x, cos x) into (sin(x + a), cos(x + a)).

demo_rotation: every p at once

CODE

```
for k in (1, 2, 5, 20):
    R = shift_rotation(k, d)
    err = np.abs(PE[k:100 + k] - PE[:100] @ R).max()
    dots = (PE[:100] * PE[k:100 + k]).sum(axis=1)
    print(f"{k:>4}{err:>32.2e}{dots.min():>28.4f}{dots.max():>10.4f}")
```

- `PE[k:100 + k] - PE[:100] @ R` — 100 shifted rows against 100 rotated rows, one matrix product.
- `(PE[:100] * PE[k:100 + k]).sum(axis=1)` — 100 dot products at once: multiply element by element, sum each row.

demo_order_visible: positions stay, words move

CODE

```
for label, pos in [("no encoding", 0.0), ("with the encoding", PE)]:
    out, _ = multi_head_attention(X + pos, P, 2)
    out_s, _ = multi_head_attention(X[:, perm] + pos, P, 2)
    print(f"{label:<19}: largest |output(shuffled) - shuffled(output)| = {np.abs(out_s - out[:,
        perm]).max():.2e}")
```

- `X[:, perm] + pos` — the words are shuffled but pos is added in its own order: position p keeps PE[p].
- `out[:, perm]` — the original outputs, shuffled the same way, for the comparison.

Segment 2 — The toy task

ToyNet: forward and gradients

MATH

$$L = \text{mean over positions of } -\log p[y];\quad dlogits = p - onehot(y), \text{ divided by } m{\cdot}n$$

CODE

```
def forward(self, tok):
    P = self.P
    n = tok.shape[1]
    X = P["E"][tok]
    if self.pe == "sin":
        X = X + self.SIN[:n]
    if self.pe == "learned":
```

```
        X = X + P["PE"][:n]
    Q, K, V = X @ P["Wq"], X @ P["Wk"], X @ P["Wv"]
    Z, A = scaled_dot_product_attention(Q, K, V)
    p = softmax(Z @ P["Wout"] + P["bout"])
    self.cache = dict(tok=tok, X=X, Q=Q, K=K, V=V, A=A, Z=Z, p=p)
    return p
def loss(self, tok, y):
    p = self.forward(tok)
    return -np.log(np.take_along_axis(p, y[..., None], axis=-1) + 1e-12).mean()
def gradients(self, tok, y):
    P = self.P
    p = self.forward(tok); C = self.cache
    m, n = tok.shape
    dl = p.copy()
    dl[np.arange(m)[:, None], np.arange(n)[None, :], y] -= 1; dl /= m * n
    flat = lambda T: T.reshape(-1, T.shape[-1])
    g = dict(Wout=flat(C["Z"]).T @ flat(dl), bout=flat(dl).sum(axis=0))
    dQ, dK, dV = scaled_dot_product_attention_backward(dl @ P["Wout"].T, C["Q"], C["K"], C["V"],
        C["A"])
    g.update(Wq=flat(C["X"]).T @ flat(dQ), Wk=flat(C["X"]).T @ flat(dK), Wv=flat(C["X"]).T @
        flat(dV))
    dX = dQ @ P["Wq"].T + dK @ P["Wk"].T + dV @ P["Wv"].T
    g["E"] = np.zeros_like(P["E"]); np.add.at(g["E"], tok, dX)
    if self.pe == "learned":
        g["PE"] = np.zeros_like(P["PE"]); g["PE"][:n] = dX.sum(axis=0)
```

- `np.take_along_axis(p, y[..., None], axis=-1)` — picks, at every position, the probability of the right answer.
- `dl[np.arange(m)[:, None], np.arange(n)[None, :], y] -= 1` — three index arrays that broadcast to (m, n): one entry per position.
- `g["PE"][:n] = dX.sum(axis=0)` — a learned table: row p collects the gradient of every sequence's position p.

toy_accuracy

CODE

```
def toy_accuracy(net, n):
    tok, y = toy_batch(np.random.RandomState(999 + n), 500, n)
    right = net.forward(tok).argmax(axis=-1) == y
    return right[:, 1:].mean(), right.mean(axis=0)
```

- `np.random.RandomState(999 + n)` — a fixed test set per length, the same for every model.
- `right[:, 1:].mean(), right.mean(axis=0)` — two results: overall without position 0, and one number per position.

Segment 3 — The tagger

positional_attention

MATH

$$Q = X_{qk} W_q, \quad K = X_{qk} W_k, \quad V = X_v W_v$$

CODE

```
def positional_attention(Xqk, Xv, P, h, mask):
    Q, K = split_heads(Xqk @ P["Wq"], h), split_heads(Xqk @ P["Wk"], h)
    V = split_heads(Xv @ P["Wv"], h)
    Zh, A = scaled_dot_product_attention(Q, K, V, mask)
    Z = merge_heads(Zh, h)
    return Z @ P["Wo"], dict(Q=Q, K=K, V=V, A=A, Z=Z)
```

- `Xqk, Xv` — two inputs: where to look comes from Xqk, what to read from Xv. With Xqk = Xv it is Day 76's layer.
- `split_heads(...)` — Day 76: each head becomes a batch entry.

PositionalTagger.forward

MATH

$$qk: X_{qk} = E[w] + s{\cdot}PE, X_v = E[w]; \quad all: X_{qk} = X_v = E[w] + s{\cdot}PE$$

CODE

```
def forward(self, ids):
    P = self.P
    m, T = ids.shape
    self.ids, mask = ids, ids != PAD
    X = P["E"][ids]
    pos = self.PE[:T][None] if self.place != "none" else 0.0
    self.Xqk = X + pos
    self.Xv = X + pos if self.place == "all" else X
    kmask = np.repeat(mask[:, None, None, :], self.h, axis=1).reshape(m * self.h, 1, T)
    self.O, self.c = positional_attention(self.Xqk, self.Xv, P, self.h, kmask)
    self.XZ = np.concatenate([self.Xv, self.O], axis=2).reshape(m * T, -1)
    self.Hh = np.tanh(self.XZ @ P["W1"] + P["b1"])
    self.probs = softmax(self.Hh @ P["Why"] + P["by"])
    return self.probs
```

- `pos = ... if self.place != "none" else 0.0` — adding the number 0.0 leaves the word vectors unchanged.
- `self.Xv = X + pos if self.place == "all" else X` — only 'all' puts position into the values and the word's own vector.
- `np.repeat(mask[:, None, None, :], self.h, axis=1)` — one copy of the padding mask per head, then flattened like the heads.

gradients: two paths back to the word vectors

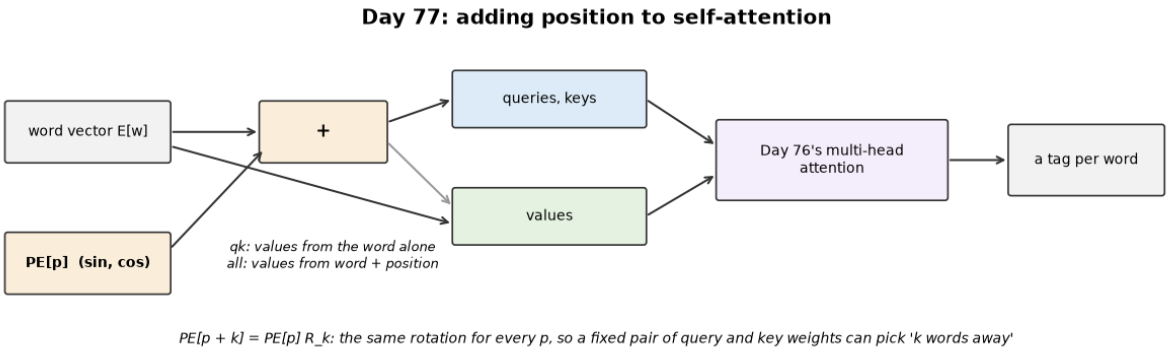
CODE

```
g["Wq"], g["Wk"] = flat(self.Xqk).T @ flat(dQ), flat(self.Xqk).T @ flat(dK)
g["Wv"] = flat(self.Xv).T @ flat(dV)
dX = dXZ[:, :, :D] + dQ @ P["Wq"].T + dK @ P["Wk"].T + dV @ P["Wv"].T
g["E"] = np.zeros_like(P["E"]); np.add.at(g["E"], self.ids, dX)
```

- `flat(self.Xqk).T @ flat(dQ)` — the query weights' gradient uses the input they actually saw (with positions).

- `dXZ[:, :, :D] + dQ @ ... + dV @ ...` — the word vector reached the output through four paths; their gradients add.
- `np.add.at(g["E"], self.ids, dX)` — Day 61: repeated words add their gradients; the fixed PE gets none.

SYSTEM DIAGRAM



Where the encoding enters the attention layer.

Segment 4 — Experiments

fit and predict

CODE

```
def predict(self, ids):
    out = np.zeros(ids.shape, dtype=int)
    for s in range(0, len(ids), 256):
        b = ids[s:s + 256]
        T = (b != PAD).sum(axis=1).max()
        out[s:s + 256, :T] = self.forward(b[:, :T]).argmax(axis=1).reshape(len(b), T)
    return out
```

- `range(0, len(ids), 256)` — batches of 256 sentences, so the (sentences x heads x T x T) weights stay small.
- `out[s:s + 256, :T] = ...` — writes this batch's predictions into the full-width array.
- `copy.deepcopy(net.P)` — in fit(): a snapshot of every weight at the best validation epoch.

offset_weights

CODE

```
def offset_weights(net, ids, offsets=(-3, -2, -1, 0, 1, 2, 3)):
    b = ids[:500]; T = (b != PAD).sum(axis=1).max(); b = b[:, :T]
    net.forward(b)
    A = net.c["A"].reshape(len(b), net.h, T, T)
    real = b != PAD
    out = np.zeros((net.h, len(offsets)))
    for j, k in enumerate(offsets):
        i = np.arange(T); i = i[(i + k >= 0) & (i + k < T)]
        ok = real[:, i] & real[:, i + k]
        out[:, j] = [A[:, hh, i, i + k][ok].mean() for hh in range(net.h)]
    return out
```

- `i = i[(i + k >= 0) & (i + k < T)]` — keeps the positions whose word k away exists in the padded array.

- `A[:, hh, i, i + k][ok]` — head hh's weight from each word i to word i + k, kept only where both are real words.

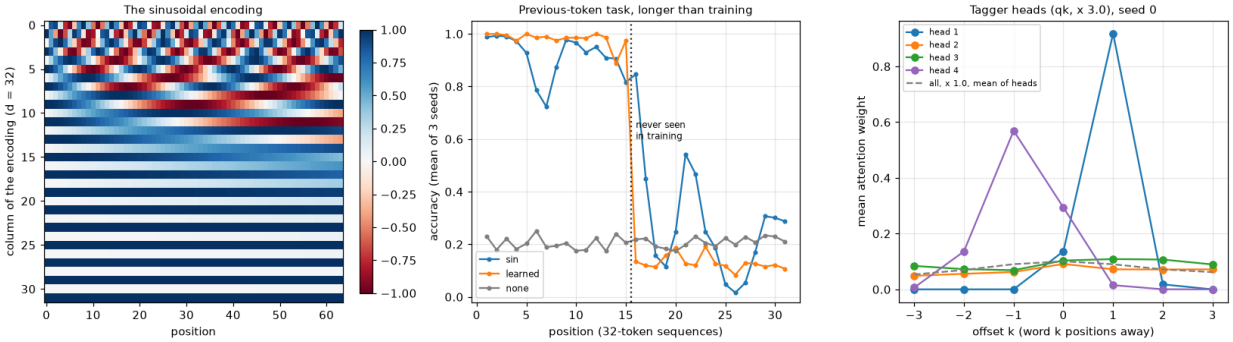
demo_shuffle: words and tags move together

CODE

```
for r in range(len(ids)):
    n = mask[r].sum(); p = rng.permutation(n)
    ids_s[r, :n], tags_s[r, :n] = ids[r, :n][p], tags[r, :n][p]
```

- `n = mask[r].sum(); p = rng.permutation(n)` — two statements on one line, separated by a semicolon.
- `ids[r, :n][p]` — the real words of sentence r, reordered by p; padding stays where it was.

REAL GRAPH



Real graph from the script: the encoding, the toy task by position, and the tagger's heads.

Quick Syntax Glossary — new or reinforced constructs

- `A[:, 0::2]` — step slicing: every second column.
- `col[:, None] / row[None, :]` — broadcasting a column against a row into a table.
- `np.take_along_axis(p, idx, axis=-1)` — pick one entry per row along an axis.
- `x + 0.0` — adding a plain number: a no-op that keeps one code path for every option.
- `a if cond else b` — a conditional expression inside an assignment.
- `np.repeat(A, h, axis=1)` — h copies along an axis.
- `copy.deepcopy(d)` — an independent copy of a dictionary of arrays.

End of Day 77 syntax reference. For the experiments and what they showed, see the Day 77 complete guide.